CodeSport

Sandya Mannarswamy

Welcome to CodeSport. This month, we take a quick look at the problem of finding the strongly connected components of a directed graph. We then discuss the problem of finding the path-sum along the root-to-leaf path of a binary tree.

A huge 'thank you' to all the readers who sent in their comments to the problems we discussed in last month's column. We also had an open problem from March -- on finding out whether a given graph with *N* nodes is connected, and to determine its best lower bound. The only question the algorithm can ask the adversary is of the form: "Does an edge exist between vertex *u* and vertex *v*?" Readers were asked to come up with the best lower bound they could establish for this algorithm using an adversary argument. Congrats to Rajeev Kumar for sending in the correct answer to this problem.

Last month's takeaway problem was from graph theory. Given a directed graph, a strongly connected component (SCC) is a set of nodes such that every pair of nodes in the SCC is mutually reachable from each other. In simple terms, SCC corresponds to reducible loops in the graph. A directed graph can contain multiple strongly connected components. One of the interesting applications of SCC is to represent the loops in the software programs we write. For example, a *for* loop that you write in your C code is internally represented as a SCC when the compiler analyses the function for optimisation opportunities. Readers were asked to come up with an algorithm to find all the strongly connected components of the graph.

Since none of the solutions I received for the SCC problem were completely correct, I am going to keep this problem open for this month also. However, in order to help those who were pretty close to getting the correct answer, I will give you a clue. For most of the graph problems, one of the first places you should look is to see if you can use the 'depth first search' (DFS) traversal to solve it. Remember that DFS on a graph will produce a forest of trees. Let us recall the definition of SCC. A strongly connected component of a graph is a maximal set of nodes, along with their edges that are strongly connected. All nodes on a SCC are inter-reachable with each other.

The SCC of a graph

If we represent each strongly connected component as a vertex, then the component graph has as its nodes each SCC in the original graph. An edge exists between two SCC nodes in the component graph if there is a directed edge from vertex *u* in one component to a vertex *v* in another component. Note that vertices *u* and *v* are NOT mutually reachable, for if there was a directed edge from *u* to *v* and from *v* to *u*, then they would have been selected as part of the same SCC. We can easily show that the component graph is a directed acyclic graph, and that each SCC component is maximal in the sense that no further vertices can be added to it. Since the SCC component graph is a DAG, it has a topological ordering and we can use it to enumerate the SCC components. Recall that we do topological sorting by listing the vertices with the longest DFS finishing time, first. Also note that if the vertices *u* and *v* are in two different SCCs, namely *U* and *V*, respectively, and if there is an edge from *u* to *v*, then F(U) is larger than F(V), where F(U) is the largest finishing time for any vertex in *U*, and F(V) is the largest DFS finishing time for any vertex in *V*. This means that we can use topological ordering to output component *V* first and then component *U*.

The transpose of a graph

Also recall that the transpose of a graph G is a graph containing the same set of vertices as in G, but an edge exists from vertex *u* to vertex *v* in the transpose graph G^T, if a directed edge exists from vertex *v* to vertex *u*. If we print the topological order of *G* in reverse order, then it is a topological order for G^T.

You should use the above facts to come up with the algorithm to find all the SCCs of graph *G*. Recall that since two vertices should be mutually reachable for them to be in a SCC, they will be present in the same tree *T* when we perform a 'depth first search' on either *G* or G^T. The strong clue is to do two DFSs, one on the original graph *G* and another on the transpose graph G^T and use